National Institute of Technology, Raipur

Summer Internship Report 2025

on development of

CU8B-I

Processor Architecture

by

Arman Sahu [23116014]
Hetram [23116043]
Niti Jain [23116067]
Pradhyumn Dwivedi [23116071]

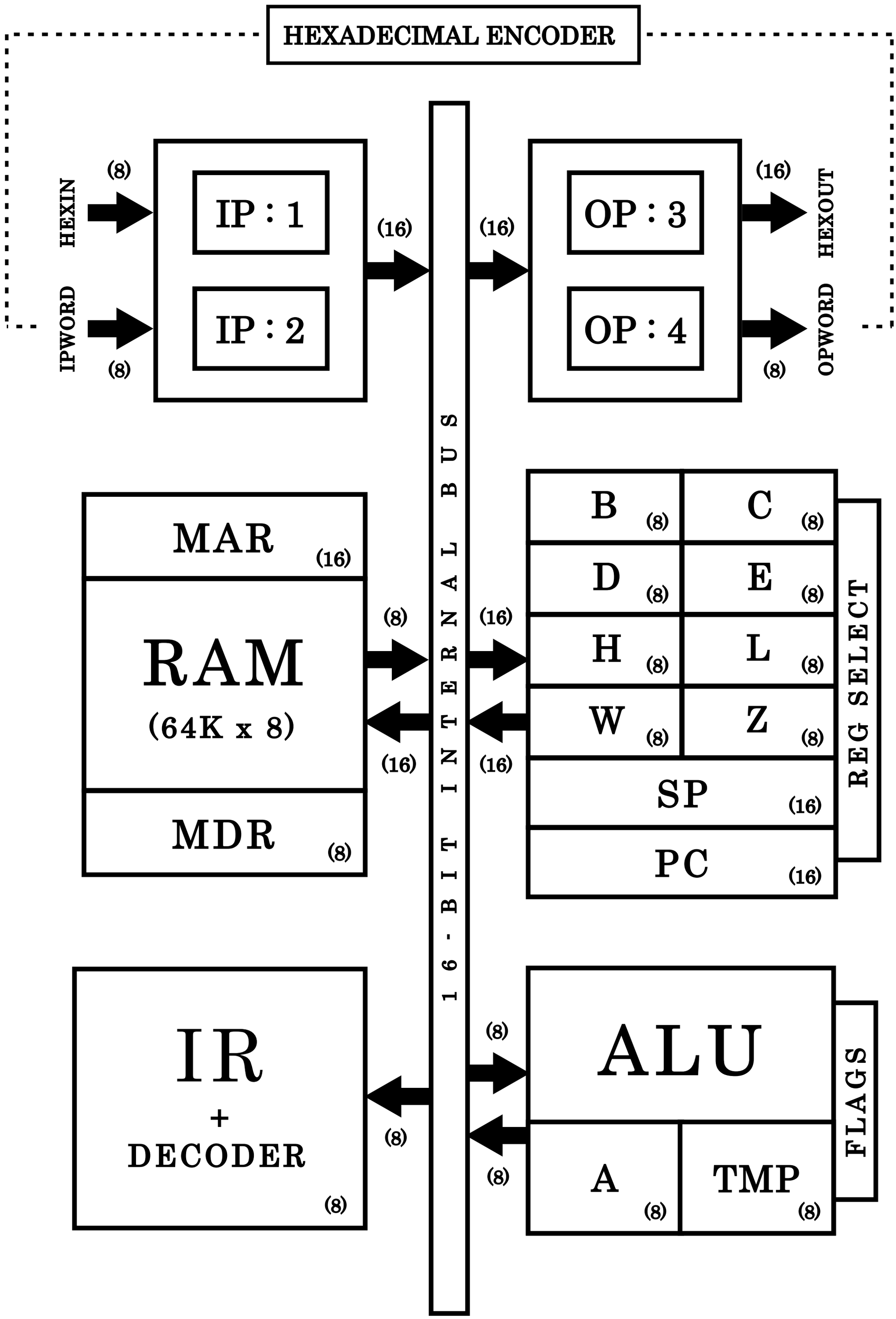
under

Prof. Shrish Verma Sir

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to Prof. Shrish Verma Sir, Dr. Bhibhudendra Acharya Sir and Snehil Sanyal Sir for their valuable guidance and constant support throughout the development of our CU8B-I Architechure. Their encouragement, timely suggestions, and technical insights played a crucial role in helping us overcome challenges and complete this project successfully.
We also sincerely thank our fellow interns who stood with us during the difficult phases, offering each other help and motivation. This project has been a great learning experience, and we are truly grateful to everyone who contributed to it.

INDEX



CU8B-I ARCHITECTURE

Components

Instruction Register (IR) & Decoder

The Instruction Register loads an 8-bit opcode from the lower nibble of the 16-bit data bus when the Load Instruction (Li) signal is active. The opcode is forwarded to the decoder/ control unit, which decodes the instruction and generates a corresponding 32-bit control word to drive execution through the datapath.

Memory Address Register (MAR)

The Memory Address Register holds a 16-bit memory address. The address is latched on the rising edge of the clock when the Load MAR (Lm) signal is activated. The data at this address is then fetched into the Memory Data Register (MDR).

RAM

The system's RAM stores 8-bit data in 64K unique memory locations, addressed using a 16-bit address line. It supports both memory read and write operations for efficient data handling.

Memory Data Register (MDR)

The MDR is an 8-bit buffer register. It receives data from the system bus during a write operation and places data onto the bus during a read operation. It acts as an interface between the memory and the processor.

Register Bank

It stores all the General Purpose Registers (GPRs). It has 8-bit (A, B, C, D, E, H, L) as well as 16-bit Registers or Register Pairs (BC, DE, HL, WZ, PC, SP) in it.

Register Pairs (BC,DE)

These are General Purpose Register pairs used to store 16-bit data values collectively. They support intermediate data. handling during program execution.

Register Pair (WZ)

These are special-purpose register pairs used to store 16-bit temporary data values. They are used by the architecture, and cannot be accessed by programmer.

Register Pair (HL)

The HL register pair serves dual purposes, It stores 16-bit memory addresses and facilitates data transfer between Input Port 1 and Output Port 3.

Program Counter (PC)

This is a Special Purpose Register of 16-bit which is used to store Address of the next instruction to be executed by the Control Unit in the Program.

Stack Pointer (SP)

This is a Special Purpose Register of 16-bit used to track the top of the stack in memory, enabling stack-based operations such as function calls and interrupts.

Arithmetic Logic Unit (ALU)

The ALU performs all arithmetic and logical operations on 8-bit data, primarily using the Accumulator and TEMP registers as operands.

Accumulator

An 8-bit register that stores the result of arithmetic and logical operations performed by the ALU.

TEMP Register

An 8-bit temporary register used for holding intermediate values or Operand Values during ALU operations.

Flag Register

A 4-bit register that stores the value of flags, indicating the current status of the Accumulator (Parity, Zero, Sign, Carry), which are essential for decision-making during program execution, especially in CALL, JUMP and RET instructions.

Input Port (1,2)

A 16-bit hexadecimal input is received via Input Port 1. Upon receiving the input, it sends a READY signal to Pin 0 of Input Port 2, enabling the input to be accessed by the Control Unit for processing (or Starting Handshake)

Output Port (3,4)

The processed data in the HL Pair can be transferred to Output Port 3, which drives the 7-Segment. Port 4 handles further data output to Hexadecimal Encoder, and its Pin 7 issues an ACKNOWLEDGE signal to the Encoder once the Control unit has successfully read the data after receiving the initial READY signal from Input Port 2.

Inbuilt Seven-Segment Decoder

The inbuilt Decoder converts the value in the Output Port 3 to valid Seven-Segment Based values for display one at a time.

Inbuilt Encoder for Hexadecimal Keyboard

The inbuilt Encoder converts a 8-bit hexadecimal value outputted by the keyboard at a time to 4-bit Hexadecimal digits in binary and after 4 Continuous Presses the complete 16-bit Hexadecimal is fed into Input Port 1 and a Ready signal is activated waiting to be Acknowledged by the CPU.

32-BIT CONTROL WORD

Et Ip Is Ds Rs1(5) Rs2(5) Lm Mr Mw Li La Ea Lt Df Ef Lc Su(4) Ei1 Ei2 Lo3 Lo4		
SIGNAL	FULL FORM	USAGE WITH ARCHITECHURE
Et	ENABLE TRANSFER	When enabled, it allows transfers to take place in Register Bank as (Rs1 = Rs2) format
Ip	INCREMENT PC	When enabled, PC gets incremented by 1
Is	INCREMENT SP	When enabled, SP gets incremented by 1
Ds	DECREMENT SP	When enabled, SP gets decremented by 1
Rs1	REGISTER SELECT 1	It has 5 select bits, the selected register/wire's value gets assigned by Rs2
Rs2	REGISTER SELECT 2	It has 5 select bits, the selected register/wire's value is assigned to Rs1
Lm	LOAD MAR	When enabled, contents of internal bus is loaded in MAR, and value at this address is loaded in MDR
Mr	MEMORY READ	When enabled, the contents of the MDR are put on lower nibble of internal bus
Mw	MEMORY WRITE	When enabled, the contents of the internal bus is loaded in MDR and at the address stored in MAR also
Li	LOAD IR	When enabled, lower nibble of the internal bus is loaded in IR
La	LOAD ACCUMULATOR	When enabled, lower nibble of internal bus is loaded in register A
Ea	ENABLE ACCUMULATOR	When enabled, the contents of register A is put on lower nibble of internal bus
Lt	LOAD TEMP	When enabled, the lower nibble of internal bus is loaded in register TMP
Df	DISABLE EFFECT ON FLAG	When enabled, it disables the effect of arithmetic operations on the value of flag register
Ef	ENABLE FLAG	When enabled, the contents of flag register is put on the lower nibble of internal bus
Lc	LOAD CARRY	When enabled, the value of carry flag is set to LSB of lower nibble of internal bus
Su	SELECT OPERATION	It is a 4 bit value, used for selecting required operation in ALU
Ei1	ENABLE INPUT PORT 1	When enabled, the contents of INPUT PORT 1 is put on the internal bus
Ei2	ENABLE INPUT PORT 2	When enabled, the contents of INPUT PORT 2 is put on the lower nibble of internal bus
Lo3	LOAD OUTPUT PORT 3	When enabled, it loads the content of internal bus in OUTPUT PORT 3
Lo4	LOAD OUTPUT PORT 4	When enabled, it loads the content of lower nibble of internal bus in OUTPUT PORT 4

REGISTER SELECT TABLE			
Rs1	Reg/Wire	Rs2	Assigned
00000	B	00000	B
00001	C	00001	C
00010	D	00010	D
00011	E	00011	E
00100	H	00100	H
00101	L	00101	L
00110	W	00110	W
00111	Z	00111	Z
01000	PC(h)	01000	PC(h)
01001	PC(l)	01001	PC(l)
01010	BC	01010	BC
01011	DE	01011	DE
01100	HL	01100	HL
01101	WZ	01101	WZ
01110	SP	01110	SP
01111	PC	01111	PC
10000	Wout	10000	Win
>10000	16'hzzzz	>10000	16'hzzzz

(RS1 = RS2)

To select specific registers from the register bank, two select lines Rs1 and Rs2 are used:

Rs1 (Register Select 1) specifies the destination Reg/Wire, i.e., the Reg/Wire to which the value is to be assigned. Rs2 (Register Select 2) specifies the source Reg/Wire, i.e., the Reg/Wire from which the value is to be taken. By default Rs1 is set to Wout (Out To Bus). Rs2 is set to Win (In from Bus).

Operations Available

ALU OPERATION TABLE	
Su(4)	OP
0000	SUB
0001	ADD
0010	COMP
0011	AND
0100	OR
0101	EXOR
0110	RAL
0111	RAR
1000	RLC
1001	RRC
1010	INR
1011	DCR

The operation to be performed is selected by selecting the value of Su corresponding to the operations given in the table. Su is a 4-bit value can have 16 operations in total, but only 12 valid combinations are used in this specific case for CU8B-I Architecture. By default Su (=0000) is selected for SUB (Subtract) Operation.

Special Operations

DOUBLE BIT PATTERNS		
BIT PAIRS		OPERATION
La	Ea	DO OPERATION WITH CLOCK
Lt	Df	LOAD FLAG REGISTER
Ef	Lc	ENABLE CARRY FLAG

Do Operation & Result in Accumulator

When an operation is to be performed on the data stored in the Accumulator, both the Load Accumulator (La) and Enable Accumulator (Ea) control signals must be set high simultaneously. This allows data to be accessed and manipulated by the ALU and change the value of Register A.

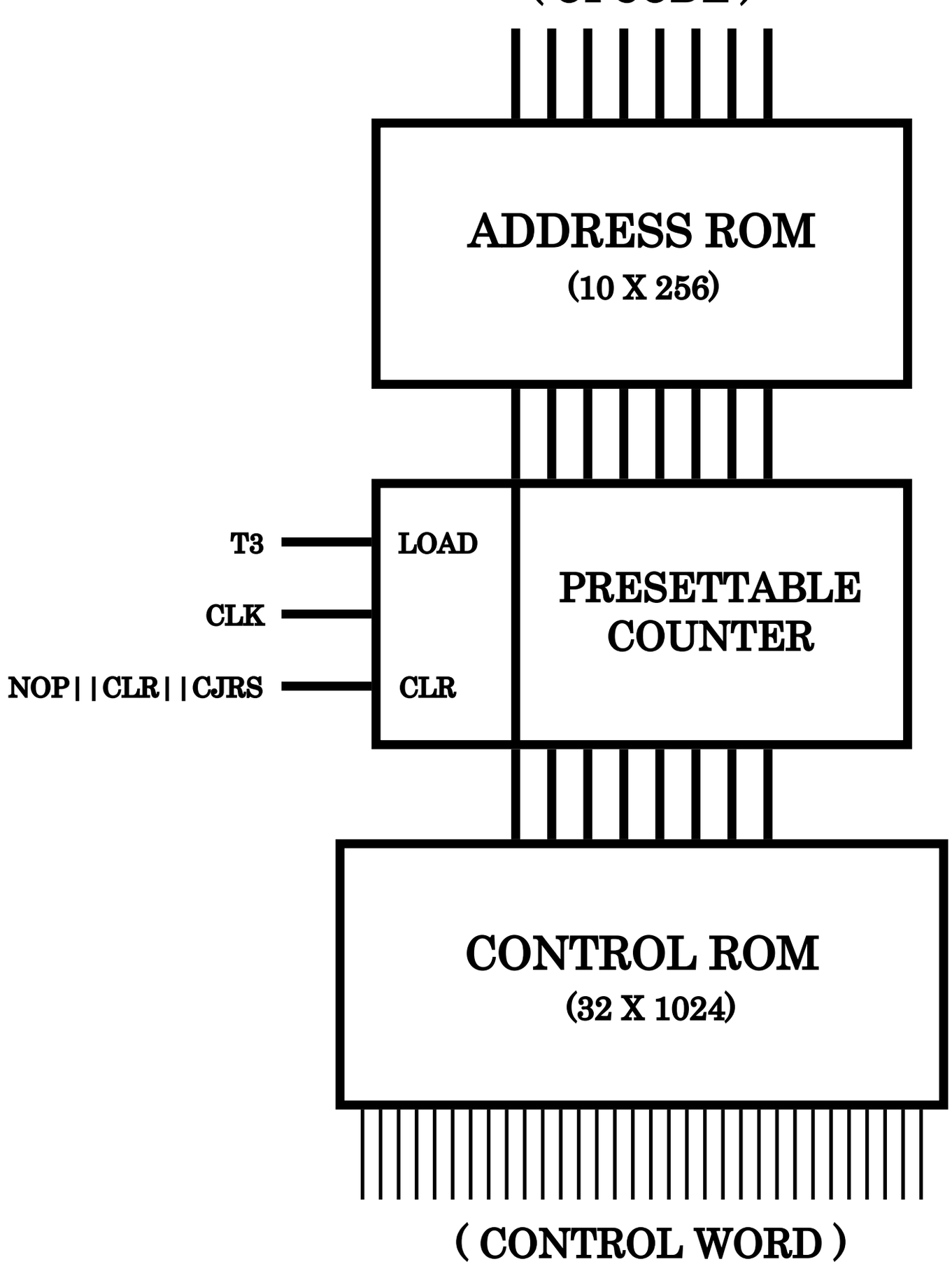
Flag Register Loading

To load data into the Flag Register, the control signals Load Temp (Lt) and Decode Flags (Df) must be asserted high concurrently. This ensures that the status flags are correctly updated based on the operation results.

Carry Flag Enable

To isolate and enable only the Carry Flag from the Flag Register, the control signals Enable Flags (Ef) and Load Carry (Lc) should both be set high. This facilitates operations that depend solely on the carry status.

Variable Machine Cycle



In reference to the control unit of SAP-1/2/3, we use a Clr signal for the Preset Counter. In CU8B-1 Architecture, we add more functionalities with it, such as Variable M/c in which the Clr signal is Or-ed with a NOP instruction capture circuit. This helps to clear the Preset Counter whenever a NOP Instruction is encountered while reading the Control ROM. values during ALU operations.

CU8B-I

INSTRUCTION SET ARCHITECHURE											
INS	OPC	INS	OPC	INS	OPC	INS	OPC	INS	OPC	INS	OPC
ACI	1	ANA C	15	CNZ	29	HLT	3d	JPE	51	MOV B,L	65
ADC A	2	ANA D	16	CP	2a	IN1	3e	JPO	52	MOV B,M	66
ADC B	3	ANA E	17	CPE	2b	IN2	3f	JZ	53	MOV C,A	67
ADC C	4	ANA H	18	CPI	2c	INR A	40	LDA	54	MOV C,B	68
ADC D	5	ANA L	19	CPO	2d	INR B	41	LXI B	55	MOV C,D	69
ADC E	6	ANA M	1a	CZ	2e	INR C	42	LXI D	56	MOV C,E	6a
ADC H	7	ANI	1b	DAD B	2f	INR D	43	LXI H	57	MOV C,H	6b
ADC L	8	CALL	1c	DAD D	30	INR E	44	LXI SP	58	MOV C,L	6c
ADC M	9	CC	1d	DAD H	31	INR H	45	MOV A,B	59	MOV C,M	6d
ADD A	0a	CM	1e	DCR A	32	INR L	46	MOV A,C	5a	MOV D,A	6e
ADD B	0b	CMA	1f	DCR B	33	INR M	47	MOV A,D	5b	MOV D,B	6f
ADD C	0c	CMC	20	DCR C	34	INX B	48	MOV A,E	5c	MOV D,C	70
ADD D	0d	CMP B	21	DCR D	35	INX D	49	MOV A,H	5d	MOV D,E	71
ADD E	0e	CMP C	22	DCR E	36	INX H	4a	MOV A,L	5e	MOV D,H	72
ADD H	0f	CMP D	23	DCR H	37	JC	4b	MOV A,M	5f	MOV D,L	73
ADD L	10	CMP E	24	DCR L	38	JM	4c	MOV B,A	60	MOV D,M	74
ADD M	11	CMP H	25	DCR M	39	JMP	4d	MOV B,C	61	MOV E,A	75
ADI	12	CMP L	26	DCX B	3a	JNC	4e	MOV B,D	62	MOV E,B	76
ANA A	13	CMP M	27	DCX D	3b	JNZ	4f	MOV B,E	63	MOV E,C	77
ANA B	14	CNC	28	DCX H	3c	JP	50	MOV B,H	64	MOV E,D	78

INS	OPC	INS	OPC	INS	OPC	INS	OPC	INS	OPC
MOV E,H	79	MOV M,D	8d	ORI	a1	RPE	b5	SUB H	c9
MOV E,L	7a	MOV M,E	8e	OUT3	a2	RPO	b6	SUB L	ca
MOV E,M	7b	MOV M,H	8f	OUT4	a3	RRC	b7	SUB M	cb
MOV H,A	7c	MOV M,L	90	POP B	a4	RZ	b8	SUI	cc
MOV H,B	7d	MVI A	91	POP D	a5	SBB A	b9	XRA A	cd
MOV H,C	7e	MVI B	92	POP H	a6	SBB B	ba	XRA B	ce
MOV H,D	7f	MVI C	93	POP PSW	a7	SBB C	bb	XRA C	cf
MOV H,E	80	MVI D	94	PUSH B	a8	SBB D	bc	XRA D	d0
MOV H,L	81	MVI E	95	PUSH D	a9	SBB E	bd	XRA E	d1
MOV H,M	82	MVI H	96	PUSH H	aa	SBB H	be	XRA H	d2
MOV L,A	83	MVI L	97	PUSH PSW	ab	SBB L	bf	XRA L	d3
MOV L,B	84	MVI M	98	RAL	ac	SBB M	c0	XRA M	d4
MOV L,C	85	ORA A	99	RAR	ad	SBI	c1	XRI	d5
MOV L,D	86	ORA B	9a	RC	ae	STA	c2		
MOV L,E	87	ORA C	9b	RET	af	STC	c3		
MOV L,H	88	ORA D	9c	RLC	b0	SUB A	c4		
MOV L,M	89	ORA E	9d	RM	b1	SUB B	c5		
MOV M,A	8a	ORA H	9e	RNC	b2	SUB C	c6		
MOV M,B	8b	ORA L	9f	RNZ	b3	SUB D	c7		
MOV M,C	8c	ORA M	a0	RP	b4	SUB E	c8		

(213 INSTRUCTIONS IN TOTAL)

Verilog Code

REGISTER BANK

```
module REGBANK(input CJR, Et, Ip, Is, Ds, Clk, Clr, [4:0] Rs1, Rs2, [15:0] Win,
output [15:0] Wout);
    reg [7:0] B=0, C=0, D=0, E=0, H, L, W=0, Z=0; reg run=0;
    reg [15:0] SP=16'h0ff0, PC=0;
    wire [15:0] Wassign;
    assign Wassign = (Rs2 == 5'b00000) ? {8'bzz,B} : 16'hz,
           Wassign = (Rs2 == 5'b00001) ? {8'bzz,C} : 16'hz,
           Wassign = (Rs2 == 5'b00010) ? {8'bzz,D} : 16'hz,
           Wassign = (Rs2 == 5'b00011) ? {8'bzz,E} : 16'hz,
           Wassign = (Rs2 == 5'b00100) ? {8'bzz,H} : 16'hz,
           Wassign = (Rs2 == 5'b00101) ? {8'bzz,L} : 16'hz,
           Wassign = (Rs2 == 5'b00110) ? {8'bzz,W} : 16'hz,
           Wassign = (Rs2 == 5'b00111) ? {8'bzz,Z} : 16'hz,
           Wassign = (Rs2 == 5'b01000) ? {8'bzz,PC[15:8]} : 16'hz,
           Wassign = (Rs2 == 5'b01001) ? {8'bzz,PC[7:0]} : 16'hz,
           Wassign = (Rs2 == 5'b01010) ? {B,C} : 16'hz,
           Wassign = (Rs2 == 5'b01011) ? {D,E} : 16'hz,
           Wassign = (Rs2 == 5'b01100) ? {H,L} : 16'hz,
           Wassign = (Rs2 == 5'b01101) ? {W,Z} : 16'hz,
           Wassign = (Rs2 == 5'b01110) ? SP : 16'hz,
           Wassign = (Rs2 == 5'b01111) ? PC : 16'hz,
           Wassign = (Rs2 == 5'b10000) ? Win : 16'hz,
           Wassign = (Rs2 > 5'b10000) ? 16'hzzzz : 16'hz;
    assign Wout = (Rs1 == 5'b10000 && Et == 1'b1) ? Wassign : 16'hzzzz;
    always @ (negedge Clk) run <= 0;
    always @ (posedge (Clk || CJR))
        begin
            if(Ip)
                begin
                    if(CJR)
                        begin
                            PC <= PC + 3;
                            run <= 1;
                        end
                    else
                        begin
                            PC <= PC + 1;
                            run <= 1;
                        end
                end
```

```
            if(Is == 1'b1) SP <= SP + 1'b1;
            if(Ds == 1'b1) SP <= SP - 1'b1;

            if(Et == 1'b1)
                begin
                    if(Rs1 == 5'b00000) B <= Wassign[7:0];
                    else if(Rs1 == 5'b00001) C <= Wassign[7:0];
                    else if(Rs1 == 5'b00010) D <= Wassign[7:0];
                    else if(Rs1 == 5'b00011) E <= Wassign[7:0];
                    else if(Rs1 == 5'b00100) H <= Wassign[7:0];
                    else if(Rs1 == 5'b00101) L <= Wassign[7:0];
                    else if(Rs1 == 5'b00110) W <= Wassign[7:0];
                    else if(Rs1 == 5'b00111) Z <= Wassign[7:0];
                    else if(Rs1 == 5'b01000) PC[15:8] <= Wassign[7:0];
                    else if(Rs1 == 5'b01001) PC[7:0] <= Wassign[7:0];
                    else if(Rs1 == 5'b01010) {B,C} <= Wassign;
                    else if(Rs1 == 5'b01011) {D,E} <= Wassign;
                    else if(Rs1 == 5'b01100) {H,L} <= Wassign;
                    else if(Rs1 == 5'b01101) {W,Z} <= Wassign;
                    else if(Rs1 == 5'b01110) SP <= Wassign;
                    else if(Rs1 == 5'b01111) PC <= Wassign;
                end
        end
endmodule
```

MAR + MEMORY + MDR

```
module MEMAR(input Lm, Mr, Mw, Clk, Clr, [15:0] Win, output [15:0] Wout);
    reg [7:0] MEM [65535:0];
    reg [15:0] MAR=0;
    reg [7:0] MDR;
    assign Wout = Mr ? {8'hz, MDR} : 16'hz;
    initial $readmemb("RAM.data", MEM);
    always @ (posedge Clk)
        begin
            if(Lm & ~Mw)
                begin
                    MAR <= Win;
                    MDR <= MEM[Win];
                end
            if(~Lm & Mw)
                begin
                    MDR <= Win[7:0];
                    MEM[MAR] <= Win[7:0];
                end
        end
endmodule
```

ALU + REG. A + REG. TEMP

```
module ALU(input La, Ea, Lt, Df, Ef, Lc, Clk, Clr, [3:0] Su, [7:0] Wlowin,
output [7:0] Wlowout, output reg [3:0] FLGS);
    reg [7:0] A=0, TMP=0;
    assign Wlowout = ({La,Ea} == 2'b01) ? A : 8'hz,
           Wlowout = ({Ef,Lc} == 2'b10) ? {4'h0, FLGS} : 8'hz,
           Wlowout = ({Ef,Lc} == 2'b11) ? {7'h0, FLGS[0]} : 8'hz;
    always @ (posedge Clk)
        begin
            if({Lt,Df} == 2'b10) TMP <= Wlowin;
            if({Lt,Df} == 2'b11) FLGS <= Wlowin[3:0];
            if({Ef,Lc} == 2'b01) FLGS[0] <= Wlowin[0];
            if({La,Ea} == 2'b10) A <= Wlowin;
            if({La,Ea} == 2'b11)
                begin
                    if(Su == 4'b0000)
                        begin
                            if({Lt,Df}==2'b00) {FLGS[0],A} = A-TMP;
                            else A = A - TMP;
                        end
                    else if(Su == 4'b0001)
                        begin
                            if({Lt,Df}==2'b00) {FLGS[0],A} = A+TMP;
                            else A = A + TMP;
                        end
                    else if(Su == 4'b0010) A = ~A;
                    else if(Su == 4'b0011) A = A & TMP;
                    else if(Su == 4'b0100) A = A | TMP;
                    else if(Su == 4'b0101) A = A ^ TMP;
                    else if(Su == 4'b0110)
                        begin
                            if({Lt,Df} == 2'b00) {FLGS[0],A} = {A,FLGS[0]};
                            else A = {A[6:0],A[7]};
                        end
                    else if(Su == 4'b0111)
                        begin
                            if({Lt,Df} == 2'b00) {FLGS[0],A} =
{A[0],FLGS[0],A[7:1]};
                            else A = {A[0],A[7:1]};
                        end
                    else if(Su == 4'b1000)
                        begin
                            if({Lt,Df} == 2'b00) {FLGS[0],A} =
{A[7],A[6:0],A[7]};
                            else A = {A[6:0],A[7]};
                        end
                    else if(Su == 4'b1001)
                        begin
                            if({Lt,Df} == 2'b00) {FLGS[0],A} =
{A[0],A[0],A[7:1]};
                            else A = {A[0],A[7:1]};
                        end
                    else if(Su == 4'b1010)
                        if({Lt,Df} == 2'b00) {FLGS[0],A} = A + 1'b1;
                        else A = A + 1'b1;
                    else if(Su == 4'b1011)
                        if({Lt,Df} == 2'b00) {FLGS[0],A} = A - 1'b1;
                        else A = A - 1'b1;
                    else {FLGS[0],A} = 0;
```

```
                    if({Lt,Df} == 2'b00)
                        begin
                            FLGS[1] = A[7];
                            FLGS[2] = ~|A;
                            FLGS[3] = ~^A;
                        end
                end
        end
endmodule
```

IR + IP(1,2) + OP(3,4) + RING COUNTER

```
module IR(input Li, Clk, Clr, [7:0] Wlow, output reg [7:0] OpCode=0);
    always @ (posedge Clk) if(Li == 1) OpCode = Wlow;
endmodule

module IP1(input Ei1, [15:0] HEXENC, output [15:0] W);
    reg [15:0] IP1;
    assign W = Ei1 ? IP1 : 16'hz;
    always @ (HEXENC) IP1 <= HEXENC;
endmodule

module IP2(input Ei2, Clk, [7:0] IPWORD, output [7:0] Wlow);
    reg [7:0] IP2;
    assign Wlow = Ei2 ? IP2 : 8'hzz;
    always @ (IPWORD) IP2 <= IPWORD;
endmodule

module OP3(input Lo3, Clk, [15:0] W, output [15:0] HEXDISP);
    reg [15:0] OP3;
    assign HEXDISP = OP3;
    always @ (posedge Clk) if(Lo3 == 1) OP3 <= W;
endmodule

module OP4(input Lo4, Clk, [7:0] Wlow, output [7:0] OPWORD);
    reg [7:0] OP4;
    assign OPWORD = OP4;
    always @ (posedge Clk) if(Lo4 == 1) OP4 <= Wlow;
endmodule

module CLKCTR(input Clk, Clr, CountClr, output T3);
    reg [7:0] Count;
    assign T3 = Count == 2;
    always @ (negedge Clk)
        begin
            if(~CountClr & ~Clr) Count <= Count + 1;
            else if(~CountClr & Clr) Count <= 0;
            else if(CountClr & ~Clr) Count <= 1;
            else if(CountClr & Clr) Count <= 1;
        end
endmodule
```

HEXADECIMAL ENCODER

```
module HEXENCODER(input Ack, Clr, [7:0] IN, output reg Ready=0, output
[15:0] OUT);
    reg R3, R2, R1, R0, C3, C2, C1, C0; reg cnd = 0;
    wire [3:0] D; reg [15:0] IP; reg [1:0] cnt=0; reg [7:0] HEXIN=0;
    assign D = {IN[5] || IN[4], IN[6] || IN[4], IN[1] || IN[0], IN[2] ||
IN[0]};
    assign OUT = IP;
 always @ (IN or Ack)
        begin
            if(IN<=8'hff)
                begin
                    if(cnt == 0)
                        begin
                            IP[15:12] <= D;
                            cnt <= cnt + 1;
                        end
                    else if(cnt == 1)
                        begin
                            IP[11:8] <= D;
                            cnt <= cnt + 1;
                        end
                    else if(cnt == 2)
                        begin
                            IP[7:4] <= D;
                            cnt <= cnt + 1;
                        end
                    else
                        begin
                            IP[3:0] <= D;
                            Ready <= 1'b1;
                            cnt <= 0;
                        end
                end
            else
                begin
                    if(Ack) Ready <= 1'b0;
                end
        end
endmodule
```

ADDRESS ROM

```
module ADDROM(output [9:0] OpCodeStart, input [7:0] OpCode);
    reg [9:0] ADDROM [255:0];
    assign OpCodeStart = ADDROM[OpCode];
    initial $readmemb("ADDROM.data", ADDROM);
endmodule
```

CONTROL ROM + PRESET COUNTER

```
module CONTROM(output [31:0] ContWord, input [9:0] OpCodeAdd);
    reg [31:0] CONTROM [1023:0];
    assign ContWord = CONTROM[OpCodeAdd];
    initial $readmemb("CONTROM.data", CONTROM);
endmodule

module PRESCNTR (output [9:0] OpCodeAdd, input [9:0] OpCodeStart, input
load, Clr, CountClr, Clk);
    reg [9:0] COUNT=0;
    assign OpCodeAdd = COUNT;
    always @ (negedge Clk)
        begin
            if (CountClr==0 & load==0) COUNT <= COUNT + 1;
            else if (CountClr==0 & load==1) COUNT <= OpCodeStart;
            else if (CountClr==1 & load==0) COUNT <= 0;
            else if (CountClr==1 & load==1) COUNT <= 0;
        end
endmodule
```

COMBINED DATAPATH

```
module DATAPATH(input Clk, Clr, CJR, [7:0] HEXIN, [31:0] ContWord, output
[3:0] FLGS, [7:0] OpCode, [15:0] HEXOUT, [15:0] W);
    wire [4:0] Rs1, Rs2; wire [3:0] Su;
    wire Et, Ip, Is, Ds, Lm, Mr, Mw, Li, La, Ea, Lt, Df, Ef, Lc, Ei1, Ei2,
Lo3, Lo4;
    assign {Et, Ip, Is, Ds, Rs1, Rs2, Lm, Mr, Mw, Li, La, Ea, Lt, Df, Ef,
Lc, Su, Ei1, Ei2, Lo3, Lo4} = ContWord;
    REGBANK rgbk(CJR, Et, Ip, Is, Ds, Clk, Clr, Rs1, Rs2, W, W);
    MEMAR mmr(Lm, Mr, Mw, Clk, Clr, W, W);
    IR ir(Li, Clk, Clr, W[7:0], OpCode);
    ALU alu(La, Ea, Lt, Df, Ef, Lc, Clk, Clr, Su, W[7:0], W[7:0], FLGS);
    wire [15:0] HEXENC;
    IP1 ip1(Ei1, HEXENC, W);
    wire Ready; wire [7:0] IPWORD, OPWORD;
    assign IPWORD = {7'b0, Ready};
    IP2 ip2(Ei2, Clk, IPWORD, W[7:0]);
    OP3 op3(Lo3, Clk, W, HEXOUT);
    OP4 op4(Lo4, Clk, W[7:0], OPWORD);
    HEXENCODER hexenc(OPWORD[7], Clr, HEXIN, Ready, HEXENC);
endmodule
```

CU8B-I MODULE (FINAL)

```
module CU8B_1(input Clk, Clr, HLT, [7:0] HEXIN, output [15:0] HEXOUT);
    wire CountClr, CALL, JMP, RET, STC, EXT, P, Z, S, C; wire [3:0] FLGS;
    wire [7:0] OpCode; wire [9:0] OpCodeStart, OpCodeAdd; wire [15:0] W;
wire [31:0] ContWord;
    assign
        load = T3,
        {P, Z, S, C} = FLGS,
        CALL = (OpCode==8'h29 & Z==1) || (OpCode==8'h2e & Z==0) ||
(OpCode==8'h28 & C==1) || (OpCode==8'h1d & C==0) || (OpCode==8'h2d & P==1)
|| (OpCode==8'h2b & P==0) || (OpCode==8'h2a & S==1) || (OpCode==8'h1e &
S==0),
        JMP = (OpCode==8'h4f & Z==1) || (OpCode==8'h53 & Z==0) ||
(OpCode==8'h4e & C==1) || (OpCode==8'h4b & C==0) || (OpCode==8'h52 & P==1)
|| (OpCode==8'h51 & P==0) || (OpCode==8'h50 & S==1) || (OpCode==8'h4c &
S==0),
        RET = (OpCode==8'hb3 & Z==1) || (OpCode==8'hb8 & Z==0) ||
(OpCode==8'hb2 & C==1) || (OpCode==8'hae & C==0) || (OpCode==8'hb6 & P==1)
|| (OpCode==8'hb5 & P==0) || (OpCode==8'hb4 & S==1) || (OpCode==8'hb1 &
S==0),
        EXT = (W[7:0]==8'h29 || W[7:0]==8'h2e || W[7:0]==8'h28 ||
W[7:0]==8'h1d || W[7:0]==8'h2d || W[7:0]==8'h2b || W[7:0]==8'h2a ||
W[7:0]==8'h1e || W[7:0]==8'h4f || W[7:0]==8'h53 || W[7:0]==8'h4e ||
W[7:0]==8'h4b || W[7:0]==8'h52 || W[7:0]==8'h51 || W[7:0]==8'h50 ||
W[7:0]==8'h4c || W[7:0]==8'hb3 || W[7:0]==8'hb8 || W[7:0]==8'hb2 ||
W[7:0]==8'hae || W[7:0]==8'hb6  || W[7:0]==8'hb5 || W[7:0]==8'hb4 ||
W[7:0]==8'hb1),
        STC = (OpCode==8'hc3 & C==1),
        CJR = (OpCodeAdd === 8'h01 && EXT) ? CALL || JMP || RET : 0,
        HLT = OpCode==8'h3d,
        CountClr = (ContWord === 32'h08400000) || CJR || STC || Clr;
    CLKCTR cc(Clk, Clr, CountClr, T3);
    ADDROM ar(OpCodeStart, OpCode);
    PRESCNTR c(OpCodeAdd, OpCodeStart, load, Clr, CountClr, Clk);
    CONTROM cr(ContWord, OpCodeAdd);
    DATAPATH dp(Clk, Clr, CJR, HEXIN, ContWord, FLGS, OpCode, HEXOUT, W);
endmodule
```

Note : This is not perfectly efficient code for CU8B-I, only for behavioral Simulation.

Example Program

10-Second Resettable Timer
(CLK - 100Hz)

- Reset Input (000B)
- Stop Input (000C)
- Restart Input (000D)

ASSEMBLY CODE

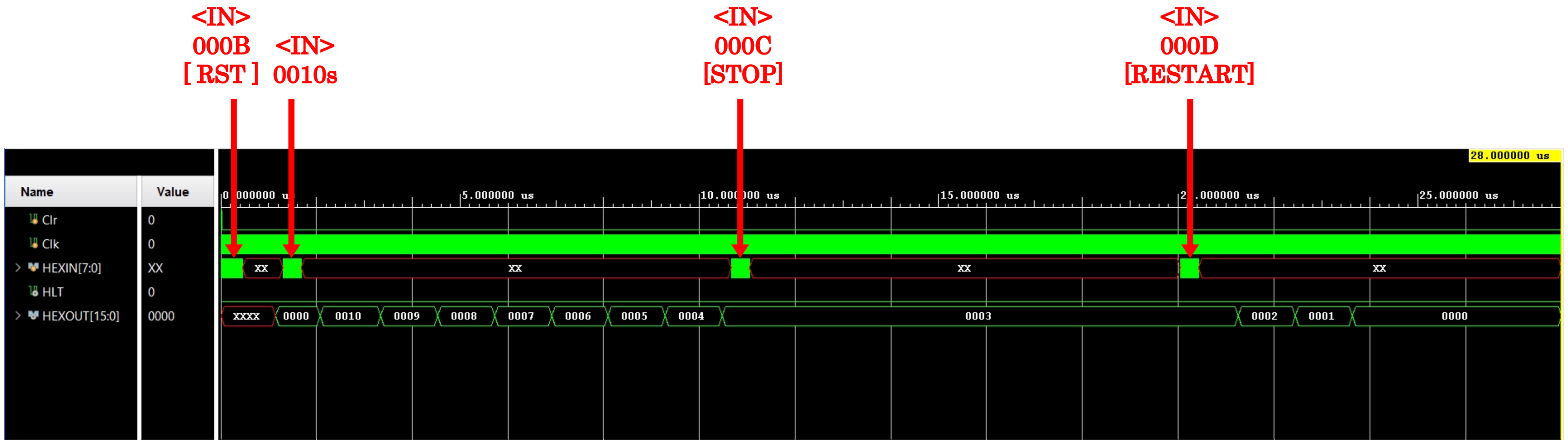
```
MVI B,0x0b
MVI C,0x0c
MVI D,0x0d
IN2
ANI 0x01
JZ 0x0006
IN1
MVI A,0x80
OUT4
MVI A,0x00
OUT4
MOV A,L
CMP B
JZ 0x0021
CMP C
JZ 0x0006
POP H
CMP D
JZ 0x0032
LXI H,0x0000
OUT3
IN2
ANI 0x01
JZ 0x0025
IN1
MVI A,0x80
OUT4
MVI A,0x00
OUT4
OUT3
MOV A,H
ADD L
```

```
JZ 0x0006
DCX H
MOV A,L
SUI 0x0f
ANI 0x0f
JNZ 0x0045
MOV A,L
ANI 0xf9
MOV L,A
MOV A,L
SUI 0xf0
ANI 0xf0
JNZ 0x0051
MOV A,L
ANI 0x5f
MOV L,A
MOV A,H
SUI 0x0f
ANI 0x0f
JNZ 0x005d
MOV A,H
ANI 0xf9
MOV H,A
IN2
ANI 0x01
JZ 0x0032
PUSH H
JMP 0x000c
```

TEST BENCH FOR GIVEN PROGRAM

```
module TESTCU8B_1;
    reg Clr, Clk; reg [7:0] HEXIN; wire HLT;
    wire [15:0] HEXOUT;
    CU8B_1 CU8B1(Clk, Clr, HLT, HEXIN, HEXOUT);
    initial
        begin
            Clk = 0;
            Clr = 1'b1;
            #7;
            Clr = 1'b0;
            #5;
        end
    always
        begin
            Clk = #5 ~Clk;
        end
    initial
        begin
            HEXIN = 8'hxx; #80; HEXIN = 8'h88; #80; HEXIN = 8'hxx; #20;
HEXIN = 8'h88; #80;
            HEXIN = 8'hxx; #20; HEXIN = 8'h88; #80; HEXIN = 8'hxx; #20;
HEXIN = 8'h21; #80;
            HEXIN = 8'hxx; #840;
            HEXIN = 8'h88; #80; HEXIN = 8'hxx; #20; HEXIN = 8'h88; #80;
HEXIN = 8'hxx; #20;
            HEXIN = 8'h84; #80; HEXIN = 8'hxx; #20; HEXIN = 8'h88; #80;
            HEXIN = 8'hxx; #9000;
            HEXIN = 8'h88; #80; HEXIN = 8'hxx; #20; HEXIN = 8'h88; #80;
HEXIN = 8'hxx; #20;
            HEXIN = 8'h88; #80; HEXIN = 8'hxx; #20; HEXIN = 8'h18; #80;
            HEXIN = 8'hxx; #9000;
            HEXIN = 8'h88; #80; HEXIN = 8'hxx; #20; HEXIN = 8'h88; #80;
HEXIN = 8'hxx; #20;
            HEXIN = 8'h88; #80; HEXIN = 8'hxx; #20; HEXIN = 8'h14; #80;
            HEXIN = 8'hxx;
        end
    initial #1000 $finish;
endmodule
```

SIMULATION OUTPUT



Note : The timescale is set to 1ns / 1ps only for simulation, for real case Clock of frequency 100Hz will be required.